



Análise do SonarQube no Projeto PokeSAL

Gabriel Castro Freitas ^{*1}, Luiz Gustavo França Barreto Silva ^{†1}, Ryan David de Oliveira Silva ^{‡1}, Orientador Pedro Arthur Melo ^{§1*}

¹ Engenharia de Software

Escola de Tecnologias

Universidade Católica do Salvador (UCSAL)

Av. Prof. Pinto de Aguiar, 2589 Pituaçu, CEP: 41740-090

Salvador/BA, Brasil

¹*{gabrielcastro.freitas@ucsal.edu.br, luizgustavofranca.silva, ryan.silva}@ucsal.edu.br*

^{1*}*{pedroam.nascimentopro.ucsal.edu.br}*

Setembro 2026

^{*}*gabrielcastro.freitas@ucsal.edu.br*

[†]*luizgustavofranca.silva@ucsal.edu.br*

[‡]*ryan.silva@ucsal.edu.br*

[§]*pedroam.nascimento@pro.ucsal.br*

1 Introdução

Este relatório apresenta uma análise dos resultados gerados pelo SonarQube no projeto **PokeSAL**. O objetivo é observar a qualidade do código a partir das principais métricas apresentadas pela ferramenta, com destaque para *Code Smells*, *Bugs*, *Vulnerabilities* e cobertura de testes (*Coverage*). Também foram considerados dados de duplicação de código, pontos de segurança e distribuição dos problemas por nível de severidade.

A análise ajuda a identificar partes do projeto que estão bem estruturadas e outras que ainda precisam de melhoria. É importante destacar que o SonarQube funciona como uma ferramenta de apoio: o fato de não encontrar bugs ou vulnerabilidades não garante que o sistema esteja totalmente livre desses problemas, mas indica que nenhuma ocorrência desse tipo foi identificada pelas regras utilizadas na análise.

2 Visão geral da análise

O SonarQube classificou o *Quality Gate* do projeto como **Passed**. Isso significa que o projeto atendeu aos critérios definidos no perfil de qualidade utilizado. Porém, a própria ferramenta também informa que a análise possui avisos, principalmente relacionados à manutenibilidade e à cobertura de testes.

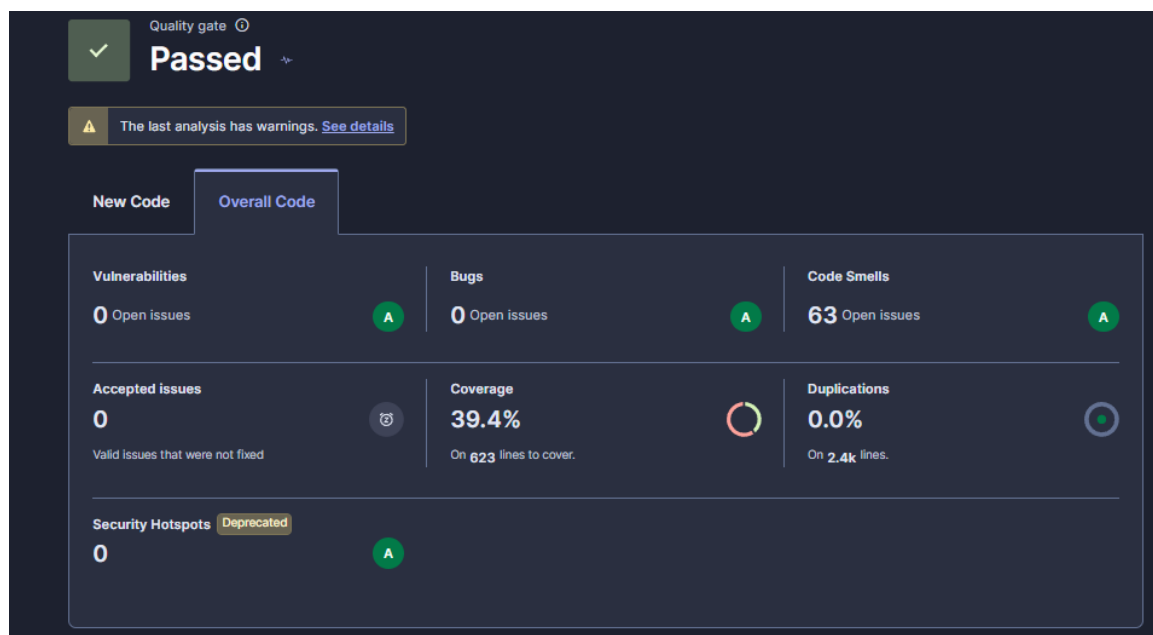


Figura 1 – Visão geral das métricas do projeto PokeSAL no SonarQube.

A Tabela 1 resume os principais resultados encontrados.

3 Análise das principais métricas

3.1 Bugs e Vulnerabilities

A análise apresentou **0 bugs** e **0 vulnerabilidades**. Esse é um resultado positivo, pois o SonarQube não encontrou problemas classificados diretamente como falhas de funcionamento ou falhas

Tabela 1 – Resumo das métricas apresentadas pelo SonarQube.

Métrica	Resultado	Interpretação
Quality Gate	Passed	Os critérios configurados no projeto foram atendidos.
Bugs	0	Nenhum bug foi identificado pela análise.
Vulnerabilities	0	Nenhuma vulnerabilidade foi identificada pelas regras utilizadas.
Code Smells	63	Existem pontos de melhoria relacionados à organização e manutenção do código.
Coverage	39,4%	Menos da metade do código analisado está coberto pelos testes.
Duplications	0,0%	Não foi detectada duplicação relevante nas aproximadamente 2,4 mil linhas analisadas.
Security Hotspots	0	Nenhum ponto de segurança que exigisse revisão manual foi indicado.
Accepted Issues	0	Nenhum problema foi marcado como aceito sem correção.

de segurança. Também não foram encontrados *Security Hotspots*.

Mesmo assim, esse resultado deve ser interpretado com cuidado. O SonarQube verifica o código de acordo com um conjunto de regras e padrões. Portanto, testes funcionais e outros tipos de teste continuam sendo necessários para verificar comportamentos que uma análise estática pode não detectar.

3.2 Code Smells

O principal ponto de atenção foi a presença de **63 Code Smells**. Esses itens não representam necessariamente erros que façam o sistema parar de funcionar, mas indicam trechos que podem dificultar manutenção, leitura ou evolução do projeto.

A tela de *Issues* mostra que, entre os 63 problemas, existem **2 itens críticos**, **59 itens classificados como Major** e **2 itens informativos**. A ferramenta estimou um esforço total de aproximadamente **1 dia e 2 horas** para tratar todos os itens apontados.

Entre os exemplos mostrados na análise estão:

- o método da classe `Main.java` apresenta complexidade cognitiva **22**, acima do limite **15** definido pela regra, indicando que o método pode estar fazendo muitas tarefas ou possuir lógica difícil de acompanhar;
- existe uma sequência de caracteres repetida cinco vezes em `Main.java`, e o SonarQube recomenda transformar esse valor em uma constante;
- há usos de `System.out` que poderiam ser substituídos por um mecanismo de *log*, deixando as mensagens da aplicação mais organizadas e mais fáceis de controlar.

Assim, a quantidade de Code Smells não indica que o projeto esteja quebrado, mas mostra que existe espaço para refatoração e melhoria da qualidade interna do código.

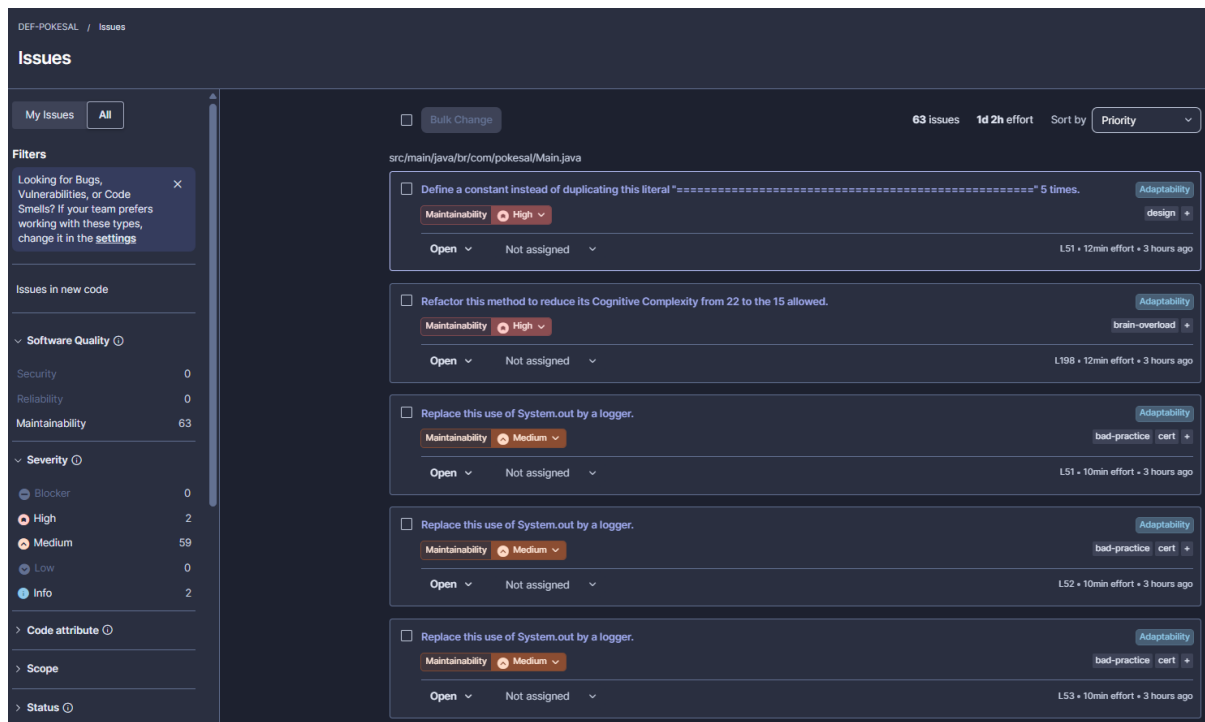


Figura 2 – Relação dos code smells mais relevantes identificados pelo SonarQube.

3.3 Cobertura de testes

A cobertura geral apresentada foi de **39,4%**, considerando **623 linhas que poderiam ser cobertas**. Desse total, **368 linhas ficaram sem cobertura**. O SonarQube também mostra **40,9% de cobertura de linhas**.

Na cobertura de condições, foram identificadas **266 condições possíveis**, das quais **171 não foram cobertas**, resultando em **35,7% de cobertura de condições**. Esse valor mostra que ainda existem muitos caminhos de decisão, como estruturas condicionais, que não são exercitados pelos testes atuais.

O detalhamento por classe ajuda a localizar os pontos mais importantes para criação de novos testes. Entre os arquivos apresentados, três chamam atenção por possuírem **0% de cobertura**: `EfeitoDeStatusGolpe.java`, `Main.java` e `SimuladorDeBatalha.java`. A classe `Main.java`, por exemplo, apresenta **151 linhas não cobertas**, sendo um dos principais candidatos para aumento da cobertura.

Outras classes apresentam cobertura parcial, como `Antidote.java` com **25,0%**, `Batalha.java` com **52,9%**, `Pokesal.java` com **53,1%** e `Treinador.java` com **63,4%**. Isso mostra que já existem testes cobrindo parte do projeto, mas ainda há espaço significativo para aumentar a quantidade de cenários testados.

3.4 Duplicação de código

A métrica de duplicação apresentou **0,0%** em aproximadamente **2,4 mil linhas analisadas**. Esse é um resultado positivo, pois indica que a ferramenta não encontrou blocos relevantes de código repetido. Evitar duplicações facilita alterações futuras e reduz a chance de corrigir um comportamento

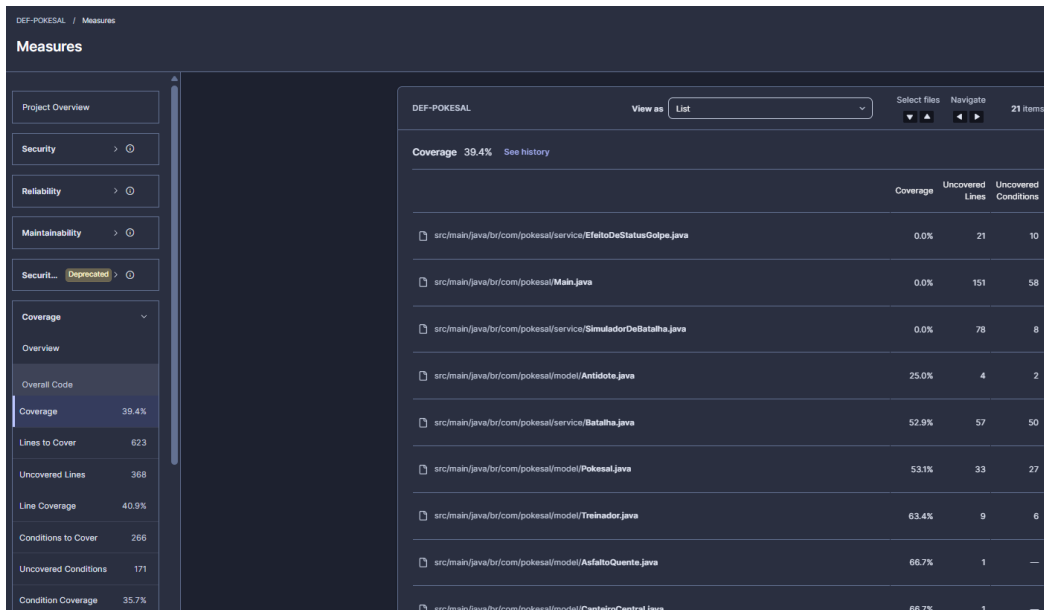


Figura 3 – Detalhamento da cobertura de testes por arquivo.

em um ponto do sistema e esquecer outro trecho semelhante.

4 Pontos positivos e pontos de melhoria

Como pontos positivos, o projeto apresentou **0 bugs**, **0 vulnerabilidades**, **0 Security Hotspots** e **0,0% de duplicação**. Além disso, o *Quality Gate* foi aprovado.

Por outro lado, existem dois pontos que merecem prioridade. O primeiro são os **63 Code Smells**, principalmente os itens críticos relacionados à complexidade do método e à repetição de valores. O segundo é a cobertura de **39,4%**, que ainda deixa uma parte considerável do código sem execução pelos testes automatizados.

Para melhorar o projeto, recomenda-se priorizar testes para as classes que aparecem com 0% de cobertura, aumentar a cobertura das condições e refatorar os Code Smells de maior severidade. Também é recomendável substituir saídas diretas com `System.out` por uma solução de *logging* e dividir métodos muito complexos em partes menores quando isso fizer sentido.

5 Conclusão

A análise do SonarQube mostra que o projeto PokeSAL apresenta bons resultados em relação a bugs, vulnerabilidades e duplicação de código. O *Quality Gate* aprovado reforça que os critérios configurados foram atendidos.

Entretanto, a análise também mostra que a qualidade pode ser melhorada. Os **63 Code Smells** indicam necessidade de refatoração em alguns trechos, enquanto a cobertura de **39,4%** revela que muitos caminhos do sistema ainda não são exercitados pelos testes. Dessa forma, a principal evolução recomendada é ampliar os testes automatizados e corrigir os problemas de manutenibilidade com maior severidade, tornando o código mais simples de manter e mais confiável para futuras alterações.